Tarea 2.2: Sistema CRUD Escolar

Documentación de Arquitectura y Funcionamiento

Alumno: Esteban de Jesús Santiago Torres

Profesor: Gerardo Cruz González

Institución: Universidad Tecnológica de la Mixteca

Contents

1 Arquitectura General del Sistema **2**

2 Módulo de Conexión: `conexion_2.py` **3**
- 2.1 Flujo de `ejecutar_sql` 4
- 2.2 Pseudocódigo de las funciones CRUD 5

3 Plantilla Base: `baseWin.py` **5**
- 3.1 Pseudocódigo de `baseWindow` 6

4 Módulo `create.py`**: Validación con Pydantic** **6**
- 4.1 Modelos de validación 7
- 4.2 Pseudocódigo de las tres ventanas Create 7

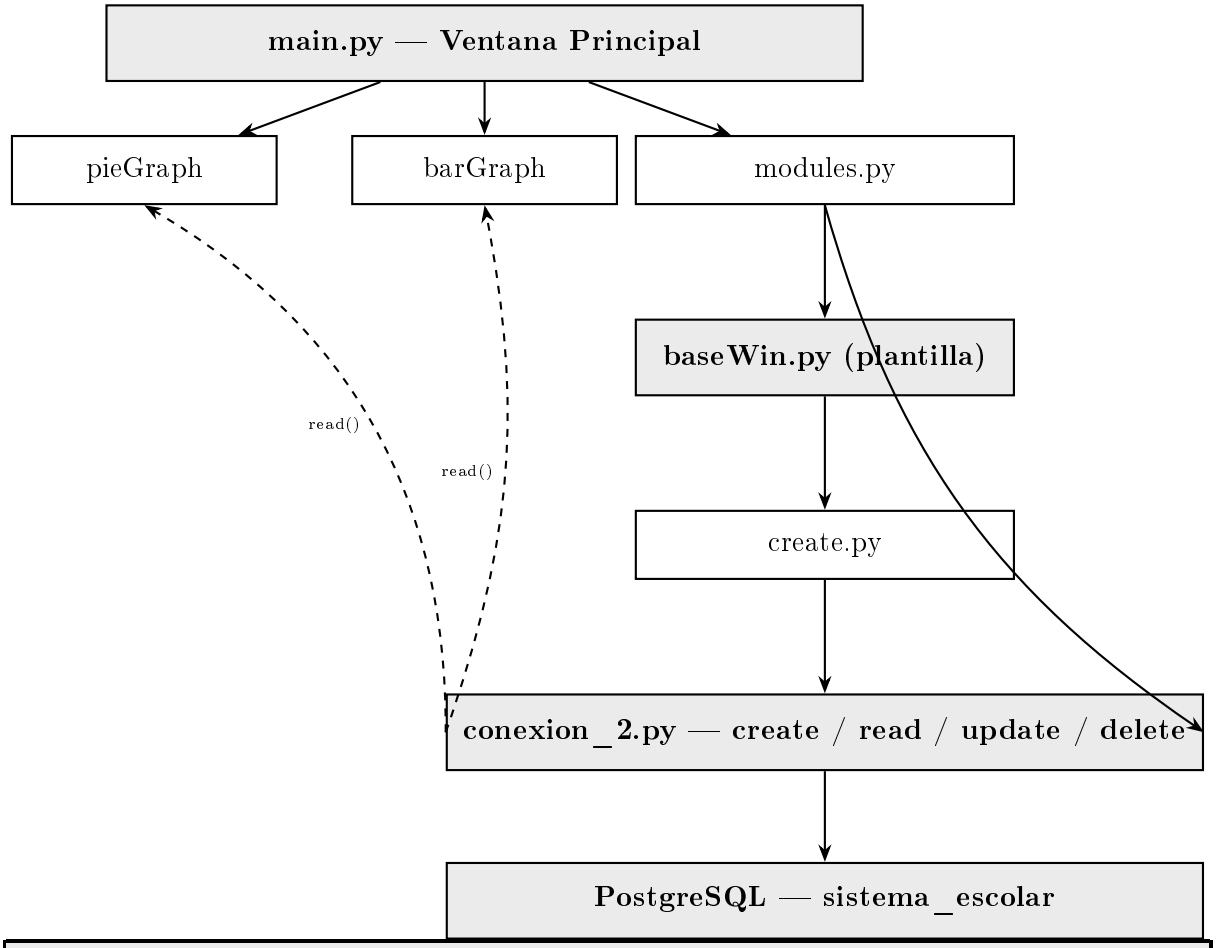
5 Módulo `modules.py`**: Ventanas CRUD** **9**
- 5.1 createWindow 9
- 5.2 readWindow (planificado) 9
- 5.3 updateWindow (planificado) 9
- 5.4 deleteWindow (planificado) 10

6 Ventana Principal: `main.py` **10**
- 6.1 Diagrama de señales 10
- 6.2 Pseudocódigo de la ventana principal 11

7 Flujo Completo: Ejemplo de uso **11**

1 Arquitectura General del Sistema

El sistema está dividido en capas que se comunican entre sí. La ventana principal (`main.py`) es el punto de entrada: muestra dos gráficas (pastel y barras) con datos leídos directamente de la base de datos, y tiene cuatro botones que abren las ventanas de *Create*, *Read*, *Update* y *Delete*. Todas estas ventanas secundarias están definidas en `modules.py` y heredan su estructura base de `baseWin.py`. La capa de datos está en `conexion_2.py`, que expone cuatro funciones (`create`, `read`, `update`, `delete`) que hablan directamente con PostgreSQL.



Las flechas discontinuas muestran que `main.py` llama a `read()` de `conexion_2` para alimentar las dos gráficas al arrancar, y también cada vez que se registra un cambio en los datos (señal `data_changed`).

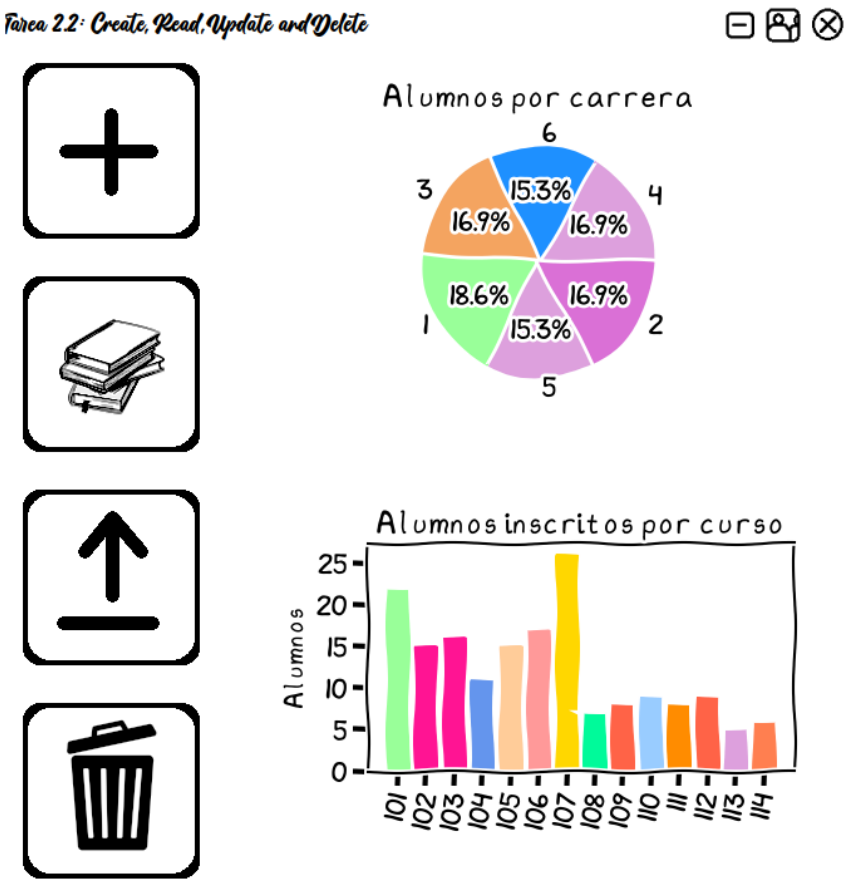
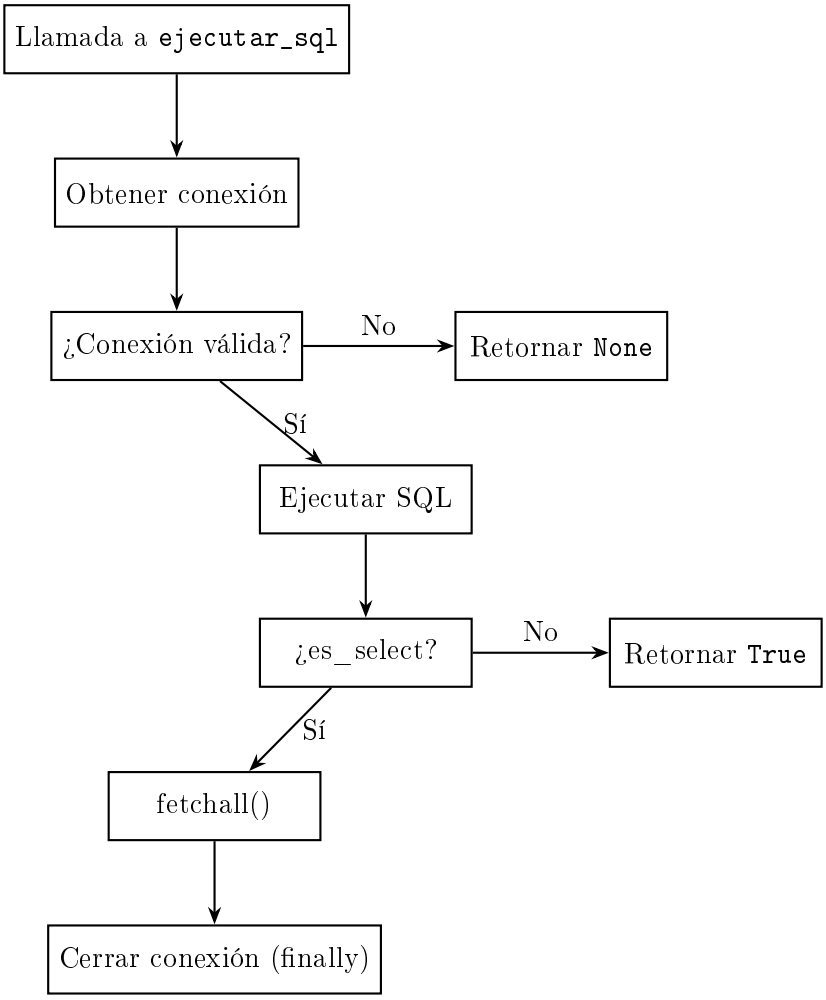


Figure 1: Ventana principal: botones CRUD (izquierda) y gráficas generadas con datos de la BD (derecha).

2 Módulo de Conexión: `conexion_2.py`

Este módulo es el único que habla con la base de datos. El resto de la aplicación nunca escribe SQL directamente; siempre pasa por aquí. La función central es `ejecutar_sql`, que abre la conexión, ejecuta el comando y la cierra siempre (con `finally`), sin importar si hubo error o no. Las cuatro funciones CRUD son simples envoltorios que construyen el SQL y llaman a `ejecutar_sql`.

2.1 Flujo de `ejecutar_sql`



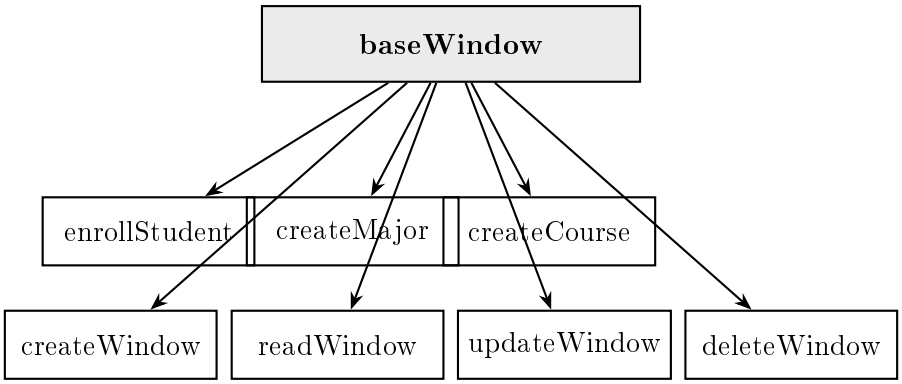
2.2 Pseudocódigo de las funciones CRUD

Algorithm 1 Operaciones CRUD

```
FUNCIÓN CREATE(tabla, datos: diccionario)
    columnas ← claves de datos
    valores ← valores de datos (como parámetros %s)
    sql ← "INSERT INTO tabla (columnas) VALUES (%s...)"
    RETORNAR ejecutar_sql(sql, valores)
FIN FUNCIÓN
FUNCIÓN READ(tabla, argumentos, adicion)
    sql ← "SELECT argumentos FROM tabla adicion"
    resultados ← ejecutar_sql(sql, es_select=True)
    SI resultados no vacío ENTONCES
        RETORNAR transponer(resultados)                ▷ filas → columnas
    FIN SI
    RETORNAR None
FIN FUNCIÓN
FUNCIÓN UPDATE(tabla, datos, condicion, params_cond)
    asignaciones ← "col=%s, col2=%s, ..."
    valores ← valores de datos + params_cond
    sql ← "UPDATE tabla SET asignaciones WHERE condicion"
    RETORNAR ejecutar_sql(sql, valores)
FIN FUNCIÓN
FUNCIÓN DELETE(tabla, condicion, params_cond)
    sql ← "DELETE FROM tabla WHERE condicion"
    RETORNAR ejecutar_sql(sql, params_cond)
FIN FUNCIÓN
```

3 Plantilla Base: `baseWin.py`

Para evitar repetir el mismo código de ventana una y otra vez, se creó la clase `baseWindow`. Cualquier ventana del sistema simplemente hereda de ella y recibe gratis: el botón de cerrar, el botón de minimizar, el título, el tamaño y la capacidad de arrastrar la ventana con el ratón (ya que la barra de título del sistema operativo está desactivada con `FramelessWindowHint`). El programador solo necesita pasar el título, el ancho y el alto como parámetros, y luego agregar sus propios controles encima.



3.1 Pseudocódigo de `baseWindow`

Algorithm 2 Clase baseWindow (plantilla de ventanas)

```
PROCEDIMIENTO __INIT__(titulo, ancho, alto, directorio)
    Quitar barra de título del SO (FramelessWindowHint)
    Cargar fuente Comic Neue desde directorio
    Crear botón Cerrar → conectar a self.close()
    Crear botón Minimizar → conectar a self.showMinimized()
    Mostrar título recibido como parámetro
FIN PROCEDIMIENTO
PROCEDIMIENTO MOUSEPRESSEVENT(evento)
    SI botón izquierdo presionado ENTONCES
        _drag_position ← posición global − posición ventana
    FIN SI
FIN PROCEDIMIENTO
PROCEDIMIENTO MOUSEMOVEEVENT(evento)
    SI botón izquierdo sostenido y _drag_position ≠ None ENTONCES
        Mover ventana a posición global − _drag_position
    FIN SI
FIN PROCEDIMIENTO
```

4 Módulo `create.py`: Validación con Pydantic

Antes de guardar cualquier dato en la base de datos, se valida con Pydantic. Esto significa que si el usuario escribe un nombre demasiado largo, deja el campo vacío, o el sistema genera una matrícula con formato incorrecto, el error se detecta *antes* de intentar insertar el registro. Si la validación falla, se muestra un mensaje en el campo de texto y no se toca la base de datos.

4.1 Modelos de validación

Modelo	Campo	Restricción
`Estudiante`	`matricula`	patrón `YYYY-NNN`, máx. 20 chars
	`nombre`	máx. 20, mín. 1 char
	`id_carrera`	entero positivo
`Carrera`	`id`	entero positivo
	`nombre`	máx. 60, mín. 1 char
`Curso`	`id`	entero ≥ 101
	`nombre`	máx. 60, mín. 1 char

4.2 Pseudocódigo de las tres ventanas Create

Algorithm 3 enrollStudent — Inscribir alumno

```
PROCEDIMIENTO ENROLL
    año ← año actual
    conteo ← read("estudiante", "count(*)", WHERE año) + 1
    matricula ← año + "-" + conteo con 3 dígitos (ej. 2025-007)
    INTENTAR
        Validar con Estudiante(matricula, nombre, id_carrera)
        create("estudiante", datos)
        Emitir señal item_created ; cerrar ventana
    EXCEPTO
        Mostrar "Nombre muy chico o muy grande" en el campo
FIN PROCEDIMIENTO
```

Algorithm 4 createMajor — Añadir carrera

```
PROCEDIMIENTO ADDMAJOR
    nuevo_id ← read("carrera", "count(*)") + 1
    INTENTAR
        Validar con Carrera(id=nuevo_id, nombre=entrada)
        create("carrera", datos)
        Emitir señal item_created ; cerrar ventana
    EXCEPTO
        Mostrar mensaje de error en el campo de texto
FIN PROCEDIMIENTO
```

Algorithm 5 createCourse — Añadir materia/curso

```
PROCEDIMIENTO ADDCOURSE
    nuevo_id ← read("curso", "count(*)") + 101 ▷ Los IDs de cursos comienzan en 101
    INTENTAR
        Validar con Curso(id=nuevo_id, nombre=entrada)
        create("curso", datos)
        Emitir señal item_created ; cerrar ventana
    EXCEPTO
        Mostrar mensaje de error en el campo de texto
FIN PROCEDIMIENTO
```

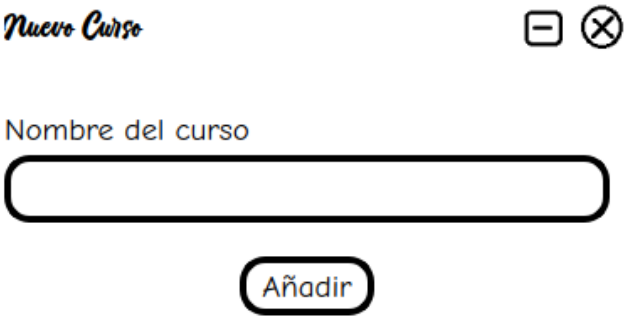


Figure 2: Formulario `createCourse`: ejemplo de ventana para añadir un nuevo curso.

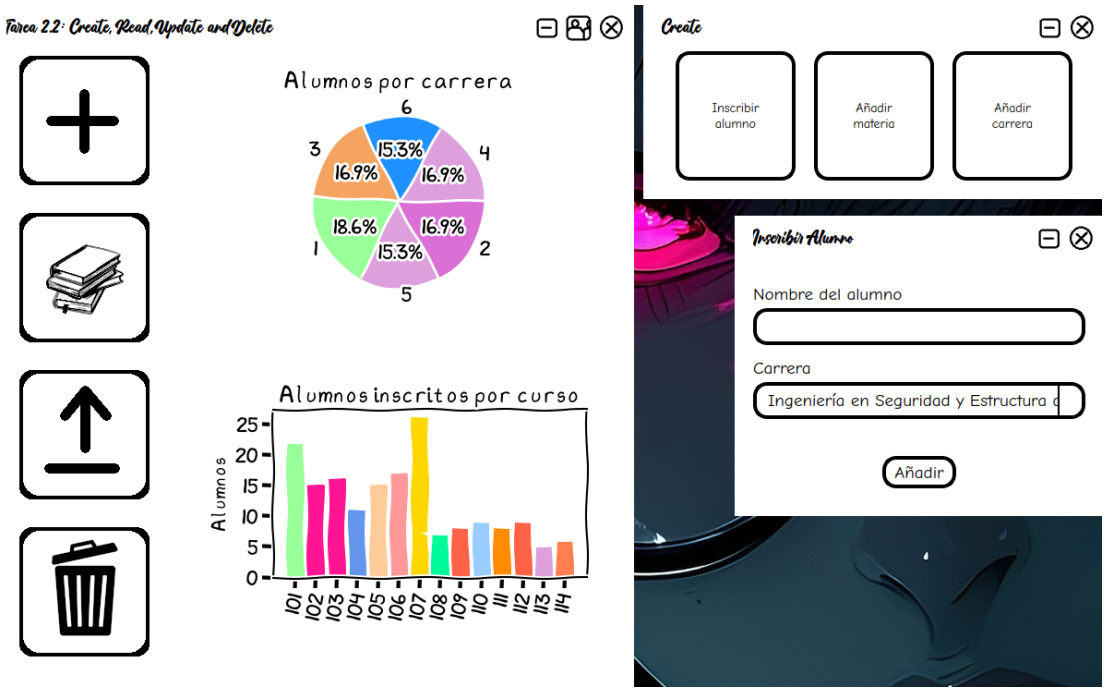


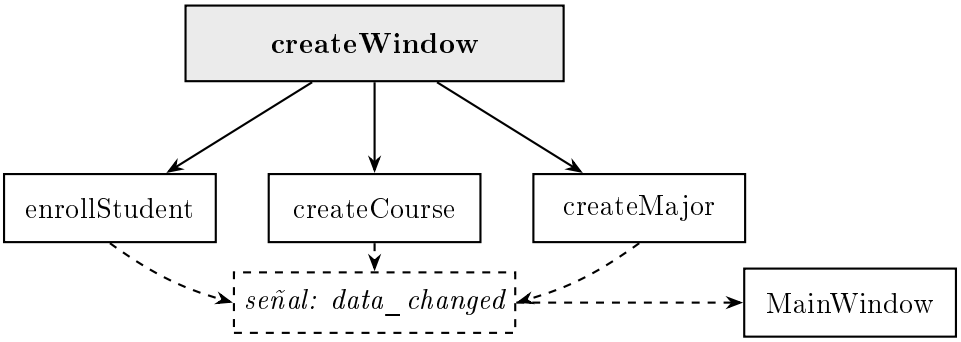
Figure 3: Flujo completo de Create: ventana principal, `createWindow` con las tres opciones e `enrollStudent` con selector de carrera.

5 Módulo `modules.py`: Ventanas CRUD

`modules.py` contiene las cuatro ventanas intermedias que el usuario ve al presionar los botones de la ventana principal. Cada una hereda de `baseWindow` y muestra un menú de opciones específicas. La ventana `createWindow` es la única completamente implementada; las demás (*Read*, *Update*, *Delete*) tienen los botones definidos pero sus acciones estaban planificadas para una segunda etapa del desarrollo.

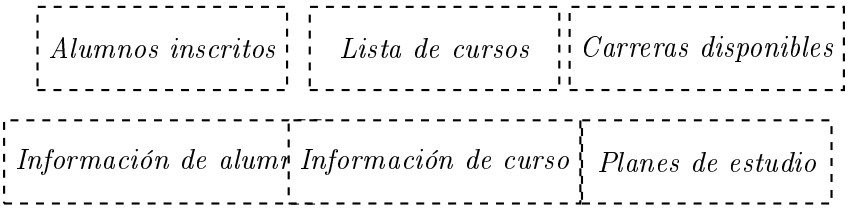
5.1 createWindow

Muestra tres botones: **Inscribir alumno**, **Añadir materia** y **Añadir carrera**. Cada botón abre la ventana correspondiente de `create.py`. Cuando cualquiera de esas ventanas termina exitosamente (señal `item_created`), `createWindow` emite su propia señal `data_changed` hacia `main.py` para que las gráficas se actualicen, y luego se cierra a sí misma.



5.2 readWindow (planificado)

Tiene seis botones planificados: **Alumnos inscritos**, **Lista de cursos**, **Carreras disponibles**, **Información de alumno**, **Información de curso** y **Planes de estudio**. La intención era que cada uno abriera una sub-ventana mostrando una tabla con los datos leídos mediante `read()` de `conexion_2`. Los botones están creados pero sus conexiones (`clicked.connect`) no se implementaron en esta entrega.



5.3 updateWindow (planificado)

Tiene tres botones planificados: **Información de alumno**, **Información de curso** y **Plan de estudios**. La idea era mostrar un formulario precargado con los datos actuales del registro seleccionado y permitir al usuario modificarlos, utilizando la función `update()` de `conexion_2`. Al igual que `createWindow`, emitiría una señal

`data_changed` al terminar para refrescar las gráficas (la línea está comentada en `main.py`).

Información de alumno | *Información de curso* | *Plan de estudios*

5.4 deleteWindow (planificado)

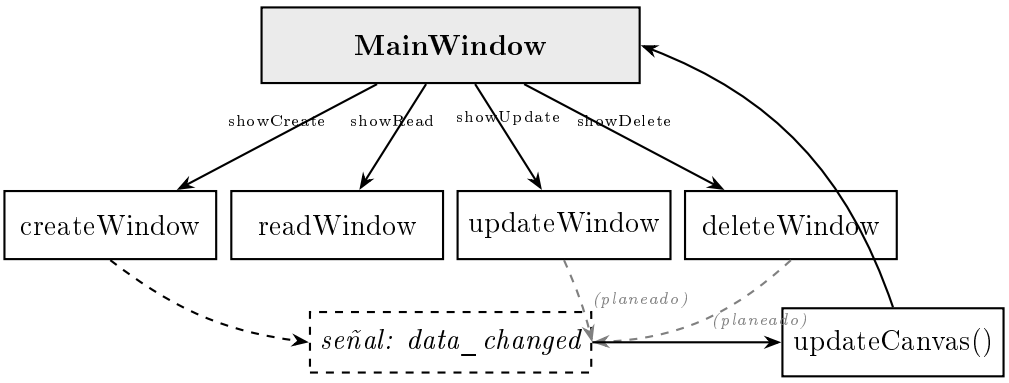
Tiene tres botones planificados: **Dar alumno de baja**, **Eliminar curso** y **Eliminar carrera**. El flujo planeado era pedir al usuario el identificador del registro (matrícula o ID), confirmar la acción, y ejecutar `delete()` de `conexion_2`. También emitiría `data_changed` (igualmente comentado en `main.py`).

Dar alumno de baja | *Eliminar curso* | *Eliminar carrera*

6 Ventana Principal: `main.py`

La ventana principal es el centro de la aplicación. Al abrirse, lee los datos de la base y genera dos gráficas: una de pastel con los estudiantes por carrera, y una de barras con los inscritos por curso. En el lado izquierdo tiene cuatro botones grandes con íconos que abren las ventanas CRUD. Cuando se registra algún cambio en los datos, el método `updateCanvas()` vuelve a consultar la base y redibuja ambas gráficas automáticamente.

6.1 Diagrama de señales



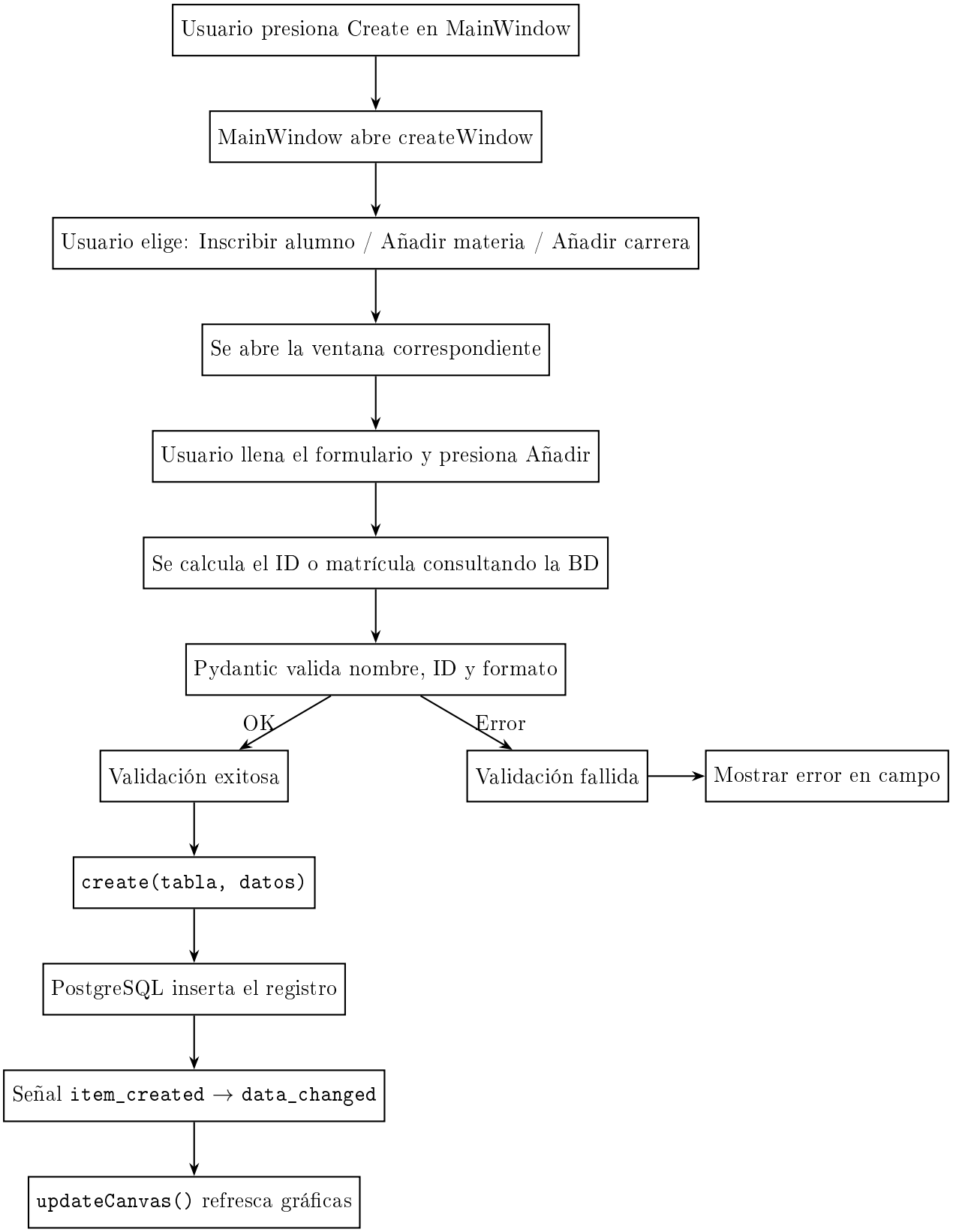
6.2 Pseudocódigo de la ventana principal

Algorithm 6 MainWindow

```
PROCEDIMIENTO __INIT__
    Crear ventana sin bordes, 700×740
    Cargar íconos y fuente Militta
    canvas0 ← pieGraph( read(carreras + conteo estudiantes) )
    canvas1 ← barGraph( read(cursos + conteo inscritos) )
    Mostrar gráficas y botones CRUD en ventana
FIN PROCEDIMIENTO
PROCEDIMIENTO SHOWCREATE
    Abrir createWindow
    Conectar señal data_changed → updateCanvas
FIN PROCEDIMIENTO
PROCEDIMIENTO UPDATECANVAS
    canvas0.update_data( read(carreras + estudiantes) )
    canvas1.update_data( read(cursos + inscritos) )
FIN PROCEDIMIENTO
```

7 Flujo Completo: Ejemplo de uso

A continuación se muestra el recorrido completo de datos desde que el usuario presiona el botón **Create** en la ventana principal hasta que el nuevo registro queda guardado en PostgreSQL y las gráficas se actualizan. El mismo patrón aplica para **Añadir carrera** y **Añadir materia**, con la diferencia de que esos flujos no necesitan calcular matrícula.



El código fuente completo del proyecto está disponible en:

https://github.com/EstebanSantiag0/CRUD

Notas de elaboración:

El proyecto fue desarrollado íntegramente en **Visual Studio Code**.

Esta documentación fue redactada en **LaTeX** con asistencia de **Inteligencia Artificial** (Claude, de Anthropic) para la generación del código fuente del documento.

El uso de IA se limitó a fines de apoyo en la elaboración del presente PDF; el diseño, la lógica y la implementación del sistema son trabajo original del alumno.